Contents

Database Optimization and Security

On delete restrict

The Customer, Product, and Order tables should not be deleted, as they are required for accounting, audits, and fraud detection.

In MySQL, ON DELETE RESTRICT is the default behaviour when defining a foreign key. This prevents a record from being deleted if it is referenced by another record.

- Once an order is created, the associated customer cannot be deleted.
- After an inventory record is created, the corresponding product cannot be deleted.
- After an order item is created, the product referenced in that item cannot be deleted, as an order must contain at least one order item.

Customer privileges

Since a user should have the privilege to remove their data from the system, when a user deletes their account, their name is set to 'deleted_user' and their phone and email are set to null. The is_deleted flag is set to true to prevent login access.

Order snapshots

The system captures a snapshot of the customer's details with each order. This ensures that changing or deleting a customer's profile will not affect past orders.

Each order saves a snapshot of the shipping address. This ensures that if a customer later updates their address, it will not affect their past orders.

An order has a separate column for the stored total because calculating it from the order items uses more resources. This also allows discounted prices to be saved.

Product soft deletion

To preserve data integrity, a product cannot be deleted if it is linked to any past orders or inventory records. Deleting it would result in the loss of historical order and inventory data.

The order item table saves a snapshot of the product price at the time of purchase. This ensures that the price on past orders is not affected by future product price updates.

Timestamp

The placed_at column in the orders table defaults to CURRENT_TIMESTAMP, so the placement time is automatically generated when a new order is created.

The inventory table uses ON UPDATE CURRENT_TIMESTAMP, which gets updated every time an update is made to a particular inventory record.

Auto increment

The customer_id, product_id, address_id, order_id, and category_id are all set as auto-incrementing primary keys. Therefore, their IDs are generated automatically and do not need to be assigned when inserting a new record.

Indexes

IDX_Product_category_map_CategoryID index
The F_GetAverageCategoryPrice function contains a WHERE pcm.CategoryID = p_CategoryID clause. By using the IDX_Product_category_map_CategoryID index, the system can jump to the exact location in the index that lists all products for the requested category, instead of iterating over the entire product category map table.

IDX_Order_PlacedAt index
Because of the IDX_Order_PlacedAt index, in P_GenerateMonthlySalesReport procedure, the system can read all the January 2024 orders, then all the February 2024 orders, in chronological sequence. Therefore, the final ORDER BY YEAR(PlacedAt), MONTH(PlacedAt) is not needed because the data is already sorted by the index.

Roles and permissions

Product Manager: Seller-side software which manages the product catalog, staff adding and product listings. This role requires permissions to view, insert, and update products, categories, and inventory.

Order Processor: An automated system that handles the checkout workflow to finalize an order and update inventory accordingly. It requires permission to read customer, address, order, and product data, and permission to update inventory quantities.

Reporting Analyst: A person or a business intelligence tool that analyzes data to find trends, track sales, and generate reports. This role requires view-only permissions for secure views of orders, products, categories, inventory, and product-category maps.

Transaction

The entire product catalog is handled as a single transaction.
Each order, including all of its line items, is also processed as an individual transaction.

JSON Array to a table

```
[
    {"productID": 1, "quantity": 1, "unitPrice": 185000.00},
    {"productID": 3, "quantity": 1, "unitPrice": 45000.00}
]
```

Part 1

```
'$[*]' COLUMNS (
        productID INT PATH '$.productID',
        qtyOrdered INT PATH '$.quantity',
        unitPrice DECIMAL(10, 2) PATH '$.unitPrice'
)
```

$[*] refers each object inside the array, one at a time.
(1). $[*] = {"productID": 1, "quantity": 1, "unitPrice": 185000.00}

```
COLUMNS (
        productID INT PATH '$.productID',
        qtyOrdered INT PATH '$.quantity',
        unitPrice DECIMAL(10, 2) PATH '$.unitPrice'
)
```

now $ refers the current object {"productID": 1, "quantity": 1, "unitPrice": 185000.00}.
$.productID -> product id of this object

part 2

```
JSON_TABLE(
      p_OrderItemsJSON,
      '$[*]' COLUMNS (
        productID INT PATH '$.productID',
        qtyOrdered INT PATH '$.quantity',
        unitPrice DECIMAL(10, 2) PATH '$.unitPrice'
      )
    ) AS jt;
```

This function returns a table called jt with the columns productID, qtyOrdered and unitPrice.

Normalization

1NF: All tables have primary keys, and each column has atomic values.
2NF: In every table, every non-key column depends on the entire primary key.
3NF: No table column is dependent on another non-key attribute (this is true for all tables except the Order table).

Overall, the schema is in 2NF.

The Order table is denormalized to preserve historical data integrity. By storing a copy of the customer's name, email, phone, and address in the Order table, the schema ensures that each order has an accurate record as it was at the time it was placed.